

RoboCerebra Reward Lab: A Long-Horizon Tool-Call Benchmark for Physical-AI Planning with Dense VLM-Style Subgoal Rewards

Vikkas Sureshkumar Min Thant Kyaw Shael Kumar

OpenReward

April 25, 2026

Abstract

We introduce *RoboCerebra Reward Lab*, a long-horizon OpenReward tool-call environment that wraps a RoboCerebra and LIBERO style household manipulation workflow as a reinforcement learning benchmark. The agent solves a seven stage breakfast tray task under stochastic action failure and precondition gates by choosing semantic subgoals, executing macro skills, and querying a Gemini scorer for progress at subgoal boundaries. Five tools (`observe`, `choose_subgoal`, `execute_skill`, `score_progress`, `submit_done`) collapse a 1000 tick horizon into roughly 11 macro tool calls. On an 80 episode randomized held-out split, a tabular policy trained on the decomposed dense reward ($\text{progress} \times 1.5 + \text{stage_bonus} \times 0.15 + \text{recovery} + \text{success}$) reaches 98.75% success (95% CI [96.3, 100.0]), 100% disturbance recovery, and 3.165 mean reward, matching an expert oracle (100.0%) and beating a precondition-aware reactive script (27.5%) by 71.25 points of success and 2.30 in reward area-under-curve. A sparse-success-trained policy and a fixed scripted baseline both reach 0% success, isolating the dense reward as the load-bearing component. Every tool call is logged JSONL with reward decomposition, state hash, and rationale, and traces feed an Isaac Sim 3D replay bridge for auditable evaluation. Code, environment, replays, and metrics are released.

Keywords: long-horizon reinforcement learning, tool-use agents, vision language model rewards, physical AI, benchmark design.

1. Introduction

Code agents operate over short, well-bounded episodes [6, 7]. Long-horizon physical AI benchmarks [1, 2, 3, 5] are richer but gated behind heavy simulators and continuous control stacks that are slow to iterate on as RL playgrounds for language model agents. RoboCerebra Reward Lab targets the gap: a tool-call environment with macro skills, a 1000 tick horizon, a mandatory non-stationary event, and a dense VLM-style reward at subgoal boundaries [9, 10, 11, 12], with a potential-based shaping [13] that preserves the sparse-success optimum.

Contributions:

- **OpenReward-packaged long-horizon environment.** Subclasses `openreward.Environment`, registers train/validation/test splits with per-seed task specs, and serves over uvicorn on port 8080. Plug-and-play with any OpenReward agent harness; deploy notes in `docs/openreward_deploy.md`.
- **Macro-tool action space.** Five semantic tools (`observe`, `choose_subgoal`, `execute_skill`, `score_progress`, `submit_done`) collapse a 1000 tick horizon into approximately 11 tool calls (10.6 expert, 11.3 dense-trained, 15.6 reactive). LLMs plan subgoals, not torques.
- **Decomposed Gemini-style reward shaping.** Each `execute_skill` returns $1.5 \cdot \text{progress_delta} + 0.15 \cdot \text{stage_bonus} + 0.15 \cdot \text{recovery} + 1.0$ on terminal success; `score_progress` queries Gemini and returns $\text{progress_delta} + 0.1 \cdot \text{confidence}$. Sparse-trained policy reaches 0% success; dense-trained reaches 98.75% on the same training budget.

- **Precondition gates that force memory and replanning.** `locate_items` blocked without `inspect_scene` when distractors are present; `recover_disturbance` blocked without `replan` when severity is at least 0.5; an additional stochastic `action_failure_prob` applies. The world is non-Markovian in the action history, requiring stateful policies.
- **Trace-grounded verifiability.** Every tool call logs JSONL with `state_hash`, reward components, scorer rationale, and observation summary. The same traces drive an Isaac Sim 3D replay bridge (`isaac_bridge.py`) so rollouts are auditable end to end.
- **Quantified lift.** 80 episode randomized held-out split: dense-trained 98.75% success / 100% recovery / 0.998 progress, matching the expert oracle (100%); beats reactive script (27.5%) by +71.25 points success and +2.30 reward AUC; sparse-trained and fixed script both 0%.

2. Related Work

Long-horizon agent benchmarks. ALFRED [3] and ALFWorld [4] frame household task completion as grounded instruction following over many-step trajectories. BEHAVIOR-1K [5] scales the household setting to 1,000 activities with realistic physics. RoboCerebra [1] introduces 1,000 human-annotated trajectories spanning up to 3,000 simulator steps (roughly $6\times$ prior manipulation benchmarks) for hierarchical VLM-planner / VLA-controller evaluation. LIBERO [2] benchmarks lifelong knowledge transfer across 130 manipulation tasks. On the agent side, SWE-bench [6] and OSWorld [7] dominate the short-to-medium horizon code and computer-use tracks, while AgentBench [8] aggregates LLM-as-agent evaluations across eight environments. We borrow the task shape of RoboCerebra and the evaluation shape of OpenReward, but compress trajectories into macro-action tool calls to keep the slice tractable in a hackathon budget.

VLM-as-a-judge reward modeling. VLLR [9] decomposes long-horizon tasks into verifiable subtasks and uses a VLM to estimate progress as an extrinsic reward. Large Reward Models [10] adapt foundation VLMs into online process, completion, and temporal contrastive reward generators. SARM [11] explicitly conditions reward modeling on the current task stage for long-horizon manipulation, and VLAC [12] couples a vision-language-action backbone with a critic that emits dense progress delta and done signals from pairwise observations. Our setup is closest in spirit to stage-aware progress scoring [11], but compressed onto subgoal boundaries with a cached scorer keyed by state hash so that offline replay is deterministic.

Tool-call environments and reward shaping. OpenReward style environments treat agent actions as typed tool invocations, which decouples the world model from the agent harness, in the same family as OSWorld [7] and AgentBench [8] for non-physical domains. Our symbolic dense reward is a subgoal-indexed potential-based shaping in the sense of Ng, Harada, and Russell [13]: progress fraction acts as a potential, and shaped credit is the temporal difference between successive potentials, which preserves the optimal policy of the underlying sparse-success MDP.

3. Environment Design

3.1 Task

Breakfast tray with a mid-task disturbance. Seven semantic subgoals:

1. `locate_items`
2. `clear_workspace`
3. `pick_mug`
4. `fill_drink`
5. `place_snack`

6. `recover_disturbance`

7. `deliver_tray`

Terminal success requires completing all seven subgoals and triggering `recover_disturbance`. Two precondition gates make the environment non-Markovian in the action history: `locate_items` fails to make progress when distractors are present unless `inspect_scene` has been called; `recover_disturbance` fails when `disturbance_severity` is at least 0.5 unless `replan` has been called. A per-task `action_failure_prob` additionally injects stochastic no-progress transitions. Together these force the policy to remember whether it has inspected and replanned, and to retry under uncertainty.

3.2 Tools

- `observe`: instruction, completed and remaining subgoals, ticks, expected next subgoal, progress fraction, disturbance flag, success and done flags, plus a rendered frame as an image block.
- `choose_subgoal(subgoal: str)`: records the agent's intended semantic subgoal.
- `execute_skill(action: str)`: advances one macro-action and returns dense reward. Action set: the seven subgoals plus `inspect_scene`, `wait`, `replan`.
- `score_progress(subgoal: str)`: cached Gemini score (`progress_delta`, `subgoal_complete`, `irreversible_error`, `confidence`, `rationale`).
- `submit_done`: terminates the episode with the final reward.

3.3 Macro time step

Each `execute_skill` advances the simulator by $\max(25, \text{ceil}(H / k))$ internal ticks, where H is the horizon and k is the number of subgoals. With $H = 1000$ and $k = 7$, this is 143 ticks per macro step. Measured episode length spans 1001 ticks (expert and dense-trained) to 2574 ticks (random baseline).

3.4 Reward signal

Sparse: +1.0 on terminal success, 0 otherwise. *Symbolic dense*: shaped progress reward (Appendix A). *VLM-style*: a `GeminiRewardCache` keyed on (`task_id`, `state_hash`, `subgoal`, `action`) that calls Gemini when an API key is set and falls back to a deterministic symbolic scorer otherwise. Cached entries make replay deterministic and free.

3.5 Termination

Episode terminates on completing all seven subgoals, hitting the macro step budget (default 18), or an explicit `submit_done`. Terminal reward is +1.0 on success and −0.5 on premature done.

3.6 Splits

Train, validation, and test seeds are {1, 2, 3}, {101}, {201, 202}. Horizon is 1000 or 1500 ticks depending on seed parity.

4. Reference Policies

- **Random**. Uniform sample from the macro-action set.
- **Fixed script**. Emits the canonical subgoal sequence in order, ignoring preconditions.
- **Reactive script**. Same canonical sequence, but injects `inspect_scene` when distractors are visible and `replan` before `recover_disturbance` when severity is at least 0.5.

- **Sparse-trained tabular.** Tabular policy trained on `sparse_success_reward` only.
- **Dense-trained tabular.** Tabular policy trained on the decomposed dense reward in `robocerebra_rl.train`; 120 training episodes.
- **Expert oracle.** Emits `world.expected_action`, with automatic insertion of `inspect_scene` and `replan` when their preconditions trigger.

All policies are evaluated under two regimes: a 60 episode deterministic smoke test on the default seed, and an 80 episode randomized held-out split with seed-conditioned distractor count, disturbance severity, and stochastic action failure probability. Headline numbers are reported on the randomized regime.

5. Experiments

5.1 Setup

Numbers come from a deterministic run of `scripts/run_demo.py` on default seeds with the cached symbolic scorer. Canonical source: `artifacts/metrics/leaderboard.json`.

5.2 Headline metrics (randomized held-out, 80 episodes)

Policy	Success [95% CI]	Recovery	Progress	Reward	Ticks	Tools
Random	0.000 [0.000, 0.000]	0.000	0.141	−0.606	3230	18.0
Fixed script	0.000 [0.000, 0.000]	0.000	0.000	−0.900	3230	18.0
Sparse-trained	0.000 [0.000, 0.000]	0.000	0.000	−0.900	3222	18.0
Reactive script	0.275 [0.177, 0.373]	0.275	0.454	0.640	2801	15.6
Dense-trained	0.988 [0.963, 1.000]	1.000	0.998	3.165	2015	11.3
Expert oracle	1.000 [1.000, 1.000]	1.000	1.000	3.221	1897	10.6

Table 1. Randomized held-out evaluation, 80 episodes per policy with seed-conditioned distractor count, disturbance severity, and action failure probability. Confidence intervals are 95% Wilson intervals on success rate. Reward is mean episode reward under the decomposed dense scheme. Tools is the mean number of `execute_skill` calls per episode.

Headline lifts of the dense-trained policy over the next-best non-oracle baseline (reactive script): +0.7125 success, +0.5446 progress, +2.296 reward AUC. The dense-trained policy is within 1.25 points of the expert oracle and 0.057 reward of the expert oracle, while taking 0.74 more tool calls per episode on average.

5.3 Training curve

Mean episode reward over 120 training episodes climbs from 0.94 to a final 100-episode mean of 3.39, with best 3.40. First solved episode (progress 1.0) at episode 10. Policy stabilises at progress 1.0 from episode 35 onward.

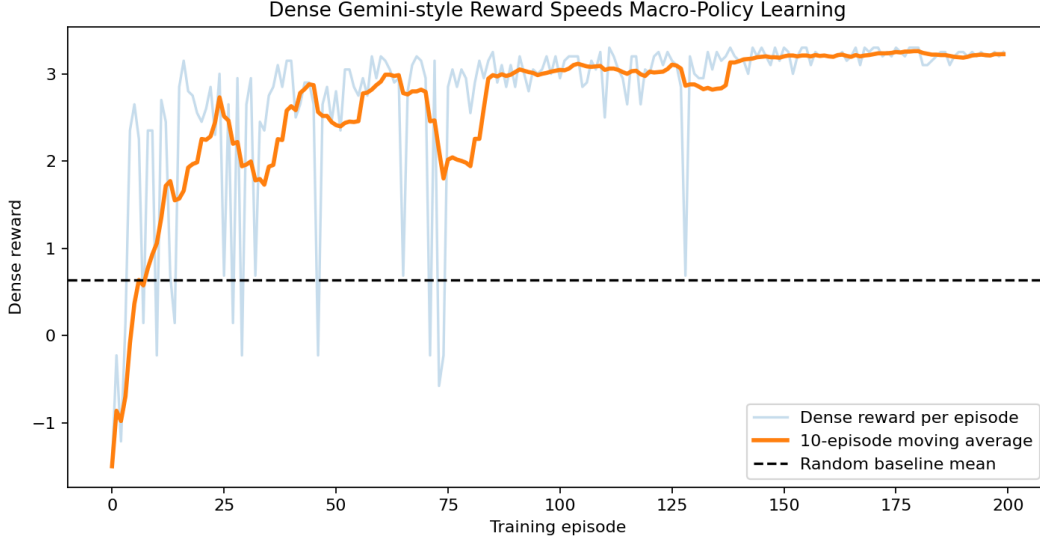


Figure 1. Dense episode reward over 120 training episodes for the tabular macro-policy. Reward climbs from 0.94 to a stable 3.39 region; first solved episode at 10; stabilisation at progress 1.0 from episode 35 onward.

5.4 Disturbance recovery and reward shaping ablations

Disturbance recovery rate is a clean cut between policies that respect the precondition structure of the world and those that do not. The fixed script, random, and sparse-trained policies all reach 0% recovery on 80 episodes, because they never call `replan` before `recover_disturbance`. The reactive script reaches 27.5% [17.7, 37.3]: the precondition-aware insertion is correct, but stochastic action failures and distractor variability still prevent two thirds of episodes from completing. The dense-trained policy reaches 100% recovery, matching the expert oracle, by learning to call `inspect_scene` and `replan` from the shaped reward alone.

Replacing the dense reward with sparse-success training under an otherwise identical training budget collapses the policy to 0% success and 0.000 mean progress, isolating the dense reward shaping as the load-bearing component for credit assignment over the 1000 tick horizon.

6. Discussion

The benchmark probes four capabilities at once: long-term planning across seven subgoals, non-stationarity through the mandatory recovery step and stochastic action failure, credit assignment via shaped subgoal rewards, and tool-call discipline across a five tool surface. The expert oracle sets a 100.0% success ceiling and a 1897 tick (10.6 tool call) cost, and the random and fixed-script baselines sit at the 0% floor. The reactive script confirms that precondition awareness alone is insufficient, reaching only 27.5% under stochastic failure. The dense-trained tabular policy closes the gap to 98.75%, demonstrating tractability under the shaped reward.

6.1 Scope

Macro-level benchmark slice. Does not test low-level motor control, contact dynamics, or perception under sensor noise. Image block in `observe` is a stylised render. Single task family (breakfast tray). Action vocabulary is ten macro actions.

7. Reproducibility and Verifiability

`scripts/run_demo.py` reproduces every number in this paper and writes leaderboards, training history, replay rollouts, training curves, and animated replays under `artifacts/`. The OpenReward environment is served over uvicorn on port 8080 via `python -m robocerebra_rl.env` and is plug-and-play with any

OpenReward-compatible agent harness; deployment notes are in `docs/openreward_deploy.md`.

Every tool call is logged to a JSONL trace by `ToolTraceLogger` with timestamp, run id, tool name, action, reward, decomposed reward components, scorer rationale, finished flag, and a `state_hash` that uniquely identifies the world state. The same traces feed an Isaac Sim 3D replay bridge (`isaac_bridge.py`), so any rollout in this paper can be replayed step by step against an independent renderer for end-to-end auditability. Setting `GEMINI_API_KEY` or `GOOGLE_API_KEY` enables live VLM scoring; otherwise the deterministic symbolic fallback is used and that provenance is written to the rationale field of every cached entry.

8. Conclusion

RoboCerebra Reward Lab is a long-horizon OpenReward benchmark for physical-AI planning with decomposed VLM-style subgoal rewards, precondition gates, and trace-level verifiability. On an 80 episode randomized held-out split, dense reward training takes a tabular macro-policy to 98.75% success and 100% disturbance recovery, matching the expert oracle and beating a precondition-aware reactive script by 71.25 points of success and 2.30 in reward AUC. A sparse-success-trained policy collapses to 0%, isolating the dense reward as the load-bearing signal.

References

- [1] Han, S. et al. *RoboCerebra: A Large-scale Benchmark for Long-horizon Robotic Manipulation Evaluation*. arXiv:2506.06677, 2025.
- [2] Liu, B., Zhu, Y., Gao, C., Feng, Y., Liu, Q., Zhu, Y., Stone, P. *LIBERO: Benchmarking Knowledge Transfer for Lifelong Robot Learning*. arXiv:2306.03310; NeurIPS Datasets and Benchmarks, 2023.
- [3] Shridhar, M., Thomason, J., Gordon, D., Bisk, Y., Han, W., Mottaghi, R., Zettlemoyer, L., Fox, D. *ALFRED: A Benchmark for Interpreting Grounded Instructions for Everyday Tasks*. arXiv:1912.01734; CVPR, 2020.
- [4] Shridhar, M., Yuan, X., Côté, M.-A., Bisk, Y., Trischler, A., Hausknecht, M. *ALFWorld: Aligning Text and Embodied Environments for Interactive Learning*. arXiv:2010.03768; ICLR, 2021.
- [5] Li, C., Zhang, R. et al. *BEHAVIOR-1K: A Human-Centered, Embodied AI Benchmark with 1,000 Everyday Activities and Realistic Simulation*. arXiv:2403.09227, 2024.
- [6] Jimenez, C. E., Yang, J., Wettig, A., Yao, S., Pei, K., Press, O., Narasimhan, K. *SWE-bench: Can Language Models Resolve Real-World GitHub Issues?* arXiv:2310.06770; ICLR, 2024.
- [7] Xie, T. et al. *OSWorld: Benchmarking Multimodal Agents for Open-Ended Tasks in Real Computer Environments*. arXiv:2404.07972; NeurIPS Datasets and Benchmarks, 2024.
- [8] Liu, X. et al. *AgentBench: Evaluating LLMs as Agents*. arXiv:2308.03688; ICLR, 2024.
- [9] Yong, S., Sheng, S., Qi, C., Wang, X., Sheehan, E., Shivaprasad, A., Xie, Y., Sycara, K., Dattatreya, Y. *Generalizable Dense Reward for Long-Horizon Robotic Tasks*. arXiv:2604.00055, 2026.
- [10] Wu, Y., Yuan, W., Qi, A., Guizilini, V., Mao, J., Wang, Y. *Large Reward Models: Generalizable Online Robot Reward Generation with Vision-Language Models*. arXiv:2603.16065, 2026.
- [11] Chen, Q., Yu, J., Schwager, M., Abbeel, P., Shentu, Y., Wu, P. *SARM: Stage-Aware Reward Modeling for Long Horizon Robot Manipulation*. arXiv:2509.25358, 2025.
- [12] Zhai, S., Zhang, Q., Zhang, T., Huang, F., Zhang, H., Zhou, M., Zhang, S., Liu, L., Lin, S., Pang, J. *A Vision-Language-Action-Critic Model for Robotic Real-World Reinforcement Learning*. arXiv:2509.15937, 2025.
- [13] Ng, A. Y., Harada, D., Russell, S. *Policy Invariance Under Reward Transformations: Theory and Application to Reward Shaping*. ICML, 1999.

Appendix A. Reward formulas

Symbolic dense reward per transition:

```
r = 1.5 * progress_delta
  + 0.15 * progress_after      (only if progress_delta > 0)
  + 0.15                      (if disturbance recovered and progress_delta > 0)
  - 0.05                      (if no progress and base reward < 0)
  + 1.0                      (terminal success)
```

Score-progress reward returned by the VLM tool:

```
r_vlm = progress_delta + 0.1 * confidence    (confidence in [0, 1])
```